Section 7 : Arrays and Vectors

Informations sur le papier

- Titre : Programmation C++
- Auteurs : `Etienne KOA`

Aperçu de la section

- Tableaux et vecteurs
- Tableaux
  - Qu'est-ce qu'ils sont
  - Pourquoi nous utilisons des tableaux
  - Déclaration et initialisation
  - Accéder aux éléments du tableau
- Tableaux multidimensionnels
- Vecteurs
  - Qu'est-ce qu'ils sont
  - Avantages par rapport aux baies
  - Déclaration et initialisation

Tableaux

- Qu'est-ce qu'un tableau ?
  - Type de données composé ou structure de données
    - Collection d'éléments
  - Tous les éléments sont du même type
  - Chaque élément est accessible directement

```
int test_score_1 {0} ;
int test_score_2 {0} ;
int test_score_3 {0} ;
int test_score_4 {0} ;
int test_score_5 {0} ;
. . .
int test_score_100 {0} ;
```

Caractéristiques

- Taille fixe
- Les éléments sont tous du même type
- Stocké de manière contiguë en mémoire

- Les éléments individuels sont accessibles par leur position ou leur index.
- Le premier élément est à l'index 0
- Le dernier élément est à la taille d'index-1
- Pas de vérification pour voir si vous êtes hors limites
- Toujours initialiser les tableaux
- Très efficace
- L'itération (bouclage) est souvent utilisée pour traiter

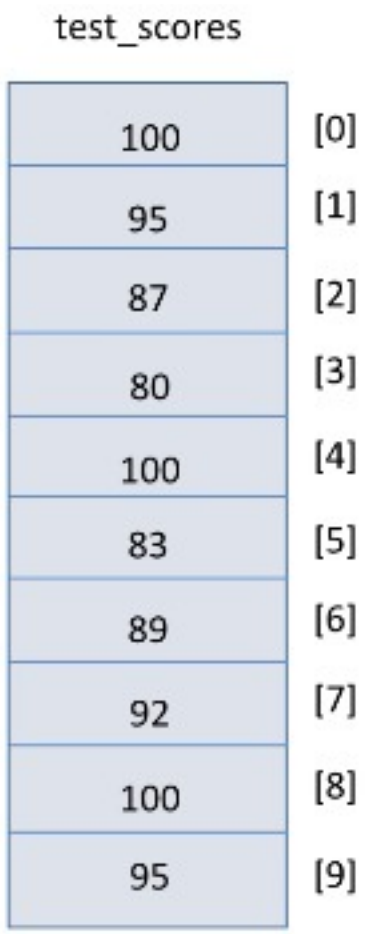

Figure 1: Tableaux

```
Element_Type array_name [constant number of elements];
```

```
int test_scores [5];


int high_score_per_level [10];

const int days_in_year {365};
double hi_temperatures [days_in_year];
```

Figure 2: Déclarer

Déclarer

Initialisation

```
Element_Type array_name [number of elements] {init list}
```

```
int test_scores [5] {100,95,99,87,88};


int high_score_per_level [10] {3,5};       // init to 3,5 and remaining to 0

const double days_in_year {365};
double hi_temperatures [days_in_year] {0}; // init all to zero


int another_array [] {1,2,3,4,5};          // size automatically calculated
```

Figure 3: Initialisation

Accéder aux éléments du tableau

Changer le contenu des éléments du tableau

Comment ça marche ?

- Le nom du tableau représente l'emplacement du premier élément du tableau (index 0).
- L'[index] représente le décalage depuis le début du tableau.
- `C++` effectue simplement une calcul pour trouver le bon élément.
- N'oubliez pas - aucune vérification des limites

```
array_name [element_index]

test_scores [1]
```

```
int test_scores [5] {100,95,99,87,88};

cout << "First score at index 0: "  << test_scores[0] << endl;
cout << "Second score at index 1: " << test_scores[1] << endl;
cout << "Third score at index 2:  " << test_scores[2] << endl;
cout << "Fourth score at index 3: " << test_scores[3] << endl;
cout << "Fifth score at index 4: "  << test_scores[4] << endl;
```

Figure 4: Accès aux éléments du tableau

```
array_name [element_index]
```

```
int test_scores [5] {100,95,99,87,88};

cin >> test_scores[0];
cin >> test_scores[1];
cin >> test_scores[2];
cin >> test_scores[3];
cin >> test_scores[4];

test_scores[0] = 90;             // assignment statement
```

Figure 5: Changer le contenu des éléments du tableau

Tableaux multidimensionnels

- Déclarer des tableaux multidimensionnels

```
Element_Type array_name [dim1_size][dim2_size]

int movie_rating [3][4];
```

Figure 6: Déclaration de tableaux multidimensionnels

- Tableaux multidimensionnels

```
const int rows {3};
const int cols {4};
int movie_rating [rows][cols];
```

movie
(second index)

reviewer
(first index)

	0	1	2	3
0	0	4	3	5
1	2	3	3	5
2	1	4	4	5

Figure 7: Tableaux multidimensionnels

- Accès aux éléments du tableau dans des tableaux multidimensionnels.
- Initialisation d'un tableau multidimensionnel

Vecteurs

- Supposons que nous souhaitions stocker les résultats des tests de mon école.
- Je n'ai aucun moyen de savoir combien d'élèves s'inscriront l'année prochaine.

+Options : + Choisissez une taille que vous ne dépasserez probablement pas et utilisez des tableaux statiques. + Utilisez un tableau dynamique tel que vectoriel.

```
cin >> movie_rating [1][2];
cout << movie_rating [1][2];
```

movie
(second index)

reviewer
(first index)

	0	1	2	3
0	0	4	3	5
1	2	3	5	5
2	1	4	4	5

Figure 8: Accès au tableau

```
int movie_rating [3][4]
{
    { 0, 4 ,3, 5},
    { 2, 3, 3, 5},
    { 1, 4, 4, 5}
};
```

	0	1	2	3
0	0	4	3	5
1	2	3	3	5
2	1	4	4	5

Figure 9: Initialisation du multidimensionnel

Qu'est-ce qu'un vecteur ?

- Conteneur dans la bibliothèque de modèles standard `C++`.
- Un tableau dont la taille peut augmenter et diminuer au moment de l'exécution.
- Fournit une sémantique et une syntaxe similaires à celles des tableaux.
- Très efficace
- Peut fournir une vérification des limites
- Peut utiliser de nombreuses fonctions intéressantes comme trier, inverser, rechercher et plus encore.

Déclarer

```
#include <vector>
using namespace std;

vector <char> vowels;

vector <int> test_scores;
```

Figure 10: Déclarer

```
vector <char> vowels (5);

vector <int> test_scores (10);
```

Figure 11: Déclarer

Initialisation

```
vector <char> vowels {'a' ,'e', 'i', 'o', 'u' };

vector <int> test_scores {100, 98, 89, 85, 93};

vector <double> hi_temperatures (365, 80.0);
```

Figure 12: Initialisation

Caractéristiques

- Taille dynamique
- Les éléments sont tous du même type
- Stocké de manière contiguë en mémoire
- Les éléments individuels sont accessibles par leur position ou leur index.
- Le premier élément est à l'index 0.
- Le dernier élément a la taille d'index-1.
- [] - pas de vérification pour voir si vous êtes hors limites.
- Fournit de nombreuses fonctions utiles qui vérifient les limites.
- Éléments initialisés à zéro
- Très efficace
- L'itération (bouclage) est souvent utilisée pour traiter

Accès aux éléments vectoriels - syntaxe du tableau

- Accès aux éléments vectoriels - syntaxe de tableau
- Accès aux éléments - syntaxe vectorielle
- Modification du contenu des éléments vectoriels - syntaxe vectorielle.
- Alors, quand poussent-ils selon les besoins ?
- Et si vous êtes hors limites ?
  - Les tableaux ne vérifient jamais les limites
  - De nombreuses méthodes vectorielles permettent de vérifier les limites
  - Une exception et un message d'erreur sont générés

```
vector_name.at(element_index)

test_scores.at(1)
```

```
vector <int> test_scores {100,95,99,87,88};

cout << "First score at index 0: "  << test_scores.at(0) << endl;
cout << "Second score at index 1: " << test_scores.at(1) << endl;
cout << "Third score at index 2:  " << test_scores.at(2) << endl;
cout << "Fourth score at index 3: " << test_scores.at(3) << endl;
cout << "Fifth score at index 4: "  << test_scores.at(4) << endl;
```

Figure 13: syntaxe du tableau

```
vector_name.at(element_index)

test_scores.at(1)
```

```
vector <int> test_scores {100,95,99,87,88};

cout << "First score at index 0: "  << test_scores.at(0) << endl;
cout << "Second score at index 1: " << test_scores.at(1) << endl;
cout << "Third score at index 2:  " << test_scores.at(2) << endl;
cout << "Fourth score at index 3: " << test_scores.at(3) << endl;
cout << "Fifth score at index 4: "  << test_scores.at(4) << endl;
```

Figure 14: syntaxe vectorielle

```
vector_name.at(element_index)
```

```
vector <int> test_scores {100,95,99,87,88};

cin >> test_scores.at(0);
cin >> test_scores.at(1);
cin >> test_scores.at(2);
cin >> test_scores.at(3);
cin >> test_scores.at(4);

test_scores.at(0) = 90;          // assignment statement
```

Figure 15: syntaxe vectorielle

```
vector_name.push_back(element)
```

```
vector <int> test_scores {100,95,99};    // size is 3

test_scores.push_back(80);               // 100, 95, 99, 80
test_scores.push_back(90);               // 100, 95, 99, 80, 90
```

Vector will automatically allocate the required space!

Figure 16: syntaxe vectorielle

```
vector <int> test_scores { 100,95 };

cin >> test_scores.at(5);

terminate called after throwing an instance of 'std::out_of_range'
what():  vector::_M_range_check: __n (which is 5) >= this->size() (which is
2)
This application has requested the Runtime to terminate it in an unusual
way.
Please contact the application's support team for more information.
```

Figure 17: syntaxe vectorielle